UGANDA CHRISTIAN UNIVERSITY

A Centre of Excellence in the Heart of Africa

FACULTY OF ENGINEERING, DESIGN AND TECHNOLOGY
DEPARTMENT OF COMPUTING AND TECHNOLOGY

STEAM TOP SELLERS: DATA MINING AND WRANGLING PROJECT REPORT

Course Unit: Data Mining and Wrangling (DSC2107)

Group: Group 7

Project Period: 13–19 October 2026

S/NO	Name in Full	Registration Number
01.	Ajak Deng Garang Deng	M25B38/031
02.	Aponi Allan Daniel	M25B38/029
03.	Bulima Ezekiel	M25B38/028
04.	Baraka Ishara Ezechiel	M25B38/041

Submission Date: 19th, October 2026

ABSTRACT

This project involved collecting, merging, cleaning, storing, and preparing public game information from Steam Top Sellers for data analysis. The main objective was to apply data mining and data wrangling techniques to real-world data using Python and MySQL.

Data were collected across seven dates, from 13 October to 19 October 2026. Six collection runs produced 500 records each, while the 17 October run produced 377 records, giving a total of 3,377 raw records. The collected information included game identifiers, rankings, categories, titles, current prices, original prices, discount percentages, release dates, review summaries, supported platforms, and collection timestamps.

Python libraries, particularly pandas, were used to merge and inspect the datasets, identify missing values, standardise fields, and prepare the data for further analysis. The initial inspection identified nine missing current prices, 2,074 missing original prices, 103 missing review summaries, and an entirely empty discount percentage column. The cleaning approach retained valid observations collected on different dates and allowed discount percentages to be calculated only when the available price information supported the calculation.

The project also prepared the cleaned data for storage in a MySQL database named steam_data, with a proposed table named steam_top_sellers_cleaned. The final database import and record count must be verified in the local MySQL environment before completion can be confirmed. The project demonstrates practical applications of web scraping, data cleaning, database management, and exploratory data analysis.

1. INTRODUCTION

Data mining and wrangling involve collecting, preparing, transforming, and analysing data to obtain useful information. Real-world datasets are rarely perfect because they may contain missing values, duplicated records, inconsistent formats, and other quality problems. Therefore, cleaning data is an important step before conducting meaningful analysis.

Steam is a digital platform where users can purchase and access computer games. Its store provides information such as game titles, prices, discounts, release dates, and user review summaries. Collecting this information over several days makes it possible to examine the available records and investigate changes in game listings.

For this project, Group 7 collected Steam Top Sellers data over seven days. The daily CSV files were combined into one dataset, inspected, and prepared for cleaning and storage in MySQL. The final dataset is intended to support further analysis of game prices, discounts, categories, and rankings.

2. PROJECT OBJECTIVES

2.1 Main Objective

To collect, merge, clean, and prepare Steam Top Sellers data for storage in MySQL and further data analysis.

2.2 Specific Objectives

The project aimed to:

1. Collect Steam game information over seven consecutive days.
2. Merge seven CSV files into one dataset.
3. identify and handle missing values, duplicated records, and inconsistent data.
4. Prepare price and discount fields for reliable analysis.
5. Store the cleaned dataset in a MySQL database.
6. Explore the dataset using descriptive statistics and visualisations.
7. Practise responsible web scraping by considering website rules and avoiding unnecessary requests.

3. METHODOLOGY

3.1 Tools and Technologies

The following tools were used or intended for the project:

- **Python:** For collecting, merging, and cleaning data.
- **pandas:** For reading CSV files, handling missing values, removing duplicates, and transforming columns.
- **Requests and BeautifulSoup:** For sending web requests and extracting information from HTML pages, where applicable.
- **Jupyter Notebook:** For organising and executing Python code in separate cells.
- **MySQL:** For storing and querying the prepared dataset.
- **MySQL Workbench:** For managing the database and checking stored records.
- **Matplotlib and Seaborn:** For creating charts and visualising patterns in the data.

3.2 Data Collection

The project collected seven CSV files covering 13–19 October 2026. The target was 500 records per daily collection. Six files contained 500 records each, while the file for 17 October contained 377 records.

Collection date	Number of records
13 October 2026	500
14 October 2026	500
15 October 2026	500
16 October 2026	500
17 October 2026	377
18 October 2026	500
19 October 2026	500
Total	**3,377**

The total number of records before cleaning was therefore 3,377. The incomplete collection on 17 October meant that the intended target of 3,500 records was not achieved.

The collected attributes included the game application ID, rank, category, title, current price, original price, discount percentage, release date, review summary, supported platforms, and scraping timestamp. A source-file column was also used during the merging process to help identify the original file for each record.

3.3 Merging the Datasets

The seven CSV files had the same main columns, so they could be combined vertically into one dataset. Python and pandas were used to read the files, combine their records, and save the merged result. The general procedure was:

1. Locate the seven CSV files.
2. Read each file into a pandas DataFrame.
3. Add a source-file field to identify the origin of each record.
4. Combine the DataFrames using pd.concat().
5. Check the total number of rows and column names.
6. Save the merged dataset as a new CSV file.

Keeping the original files unchanged was important because it allowed the merged dataset to be checked against the collected data.

4. DATA CLEANING AND MYSQL STORAGE

4.1 Data Quality Assessment

After merging the seven files, the dataset contained 3,377 records. An initial inspection identified missing values in several columns.

Column	Available records	Missing records
price	3,368	9
original_price	1,303	2,074
discount_percent	0	3,377
review_summary	3,274	103

These figures describe the dataset before the relevant cleaning and transformation steps. They should be checked again after cleaning.

4.2 Handling Missing Values

Missing values were handled according to the meaning of each column.

For the price column, the nine missing values were not automatically replaced with zero because a missing price does not necessarily mean that a game is free. These records should be checked against the source or left missing if the correct price cannot be established.

The original_price column contained 2,074 missing values. These values may indicate that an original price was not available in the collected data. However, missing information alone does not prove that a game had no discount.

All 3,377 records initially had missing values in discount_percent. Therefore, the discount field required investigation and calculation where sufficient price information was available.

The 103 missing review summaries were retained as missing unless reliable information could be obtained. Missing summaries do not necessarily indicate that a game has no reviews.

4.3 Calculating Discount Percentages

A discount percentage can be calculated when both the current price and original price are available and valid.

The formula is: **Discount Percentage = ((Original Price − Current Price) / Original Price) × 100**

For example, if a game originally costs $100 and is currently sold for $75, the discount is:

[\frac{100-75}{100}\times100=25%]

The calculation should only be performed when both prices are numeric, the original price is greater than zero, and the original price is greater than or equal to the current price. Records that do not satisfy these conditions should be investigated or left with a missing discount value.

A current price of zero may represent a free game, but this should not be confused with an unavailable price.

4.4 Removing Duplicates and Validating Records

Exact duplicate rows can be removed using pandas. However, records should not be removed simply because they have the same game title or application ID. A game may legitimately appear in several daily collections or categories.

The following code removes exact duplicate rows and checks the result:

```
print("Duplicates before cleaning:", df.duplicated().sum()) # Count exact duplicate rows.

df = df.drop_duplicates()  # Remove exact duplicate rows.

print("Rows after cleaning:", len(df))  # Check the cleaned dataset.

print("Missing values by column:")

print(df.isnull().sum())
```

Other validation tasks include checking that application IDs and ranks have appropriate formats, ensuring prices are not negative, and confirming that timestamps and dates use consistent formats. Any records that appear incorrect should be investigated rather than changed without evidence.

4.5 Storing the Dataset in MySQL

MySQL was selected to provide structured storage and allow the dataset to be queried using SQL. The intended database name was steam_data, and the table name was steam_top_sellers_cleaned.

After cleaning, the dataset should be imported into MySQL. The following queries can be used to verify the database and table:

```
SHOW DATABASES;

USE steam_data;

SHOW TABLES;

DESCRIBE steam_top_sellers_cleaned;

SELECT *
FROM steam_top_sellers_cleaned
LIMIT 10;

SELECT COUNT(*) AS total_records
FROM steam_top_sellers_cleaned;
```

To check the availability of discount information, run:

```
SELECT
    COUNT(*) AS total_records,
    COUNT(discount_percent) AS records_with_discount,
    SUM(discount_percent IS NULL) AS records_without_discount
FROM steam_top_sellers_cleaned;
```

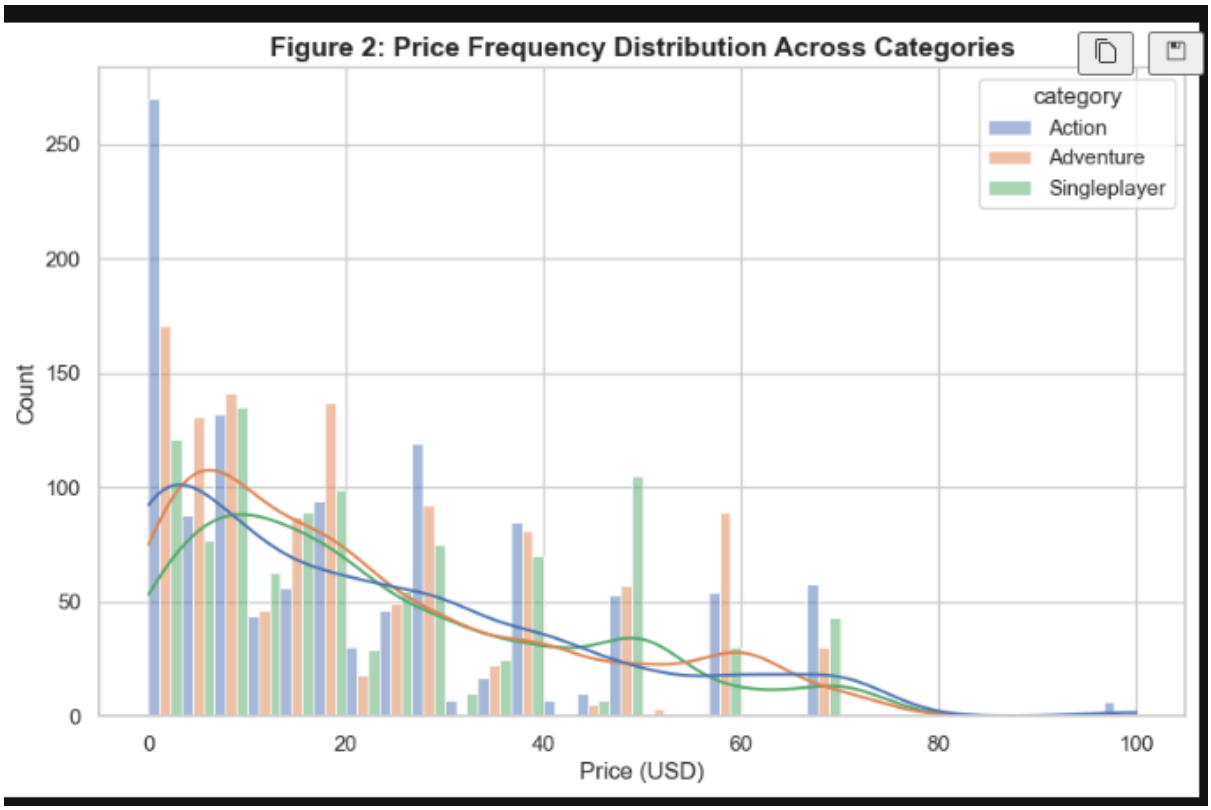
These queries help confirm that the table exists, the columns are correctly defined, and the expected records are available. The import should only be reported as successful after these checks have been completed.

5. EXPLORATORY DATA ANALYSIS AND FINDINGS

Exploratory data analysis (EDA) involves examining a dataset using summary statistics and visualisations to understand its characteristics. The cleaned Steam dataset provides opportunities to investigate prices, discounts, review summaries, free and paid games, and ranking changes.

At least six labelled charts should be created to support the analysis. The following sections describe the planned visualisations and how their findings should be interpreted.

5.1 Price Distribution and Outliers



A boxplot can be used to examine the distribution of current game prices across categories. It can show the median, spread, and potential outliers.

Very high prices may represent premium editions, bundles, or other products with different pricing structures. However, the cause of an outlier should not be assumed from the price alone.

The analysis should report the actual median, quartiles, and maximum price from the cleaned dataset. Missing prices should be excluded from calculations that require numeric values.

5.2 Free and Paid Items

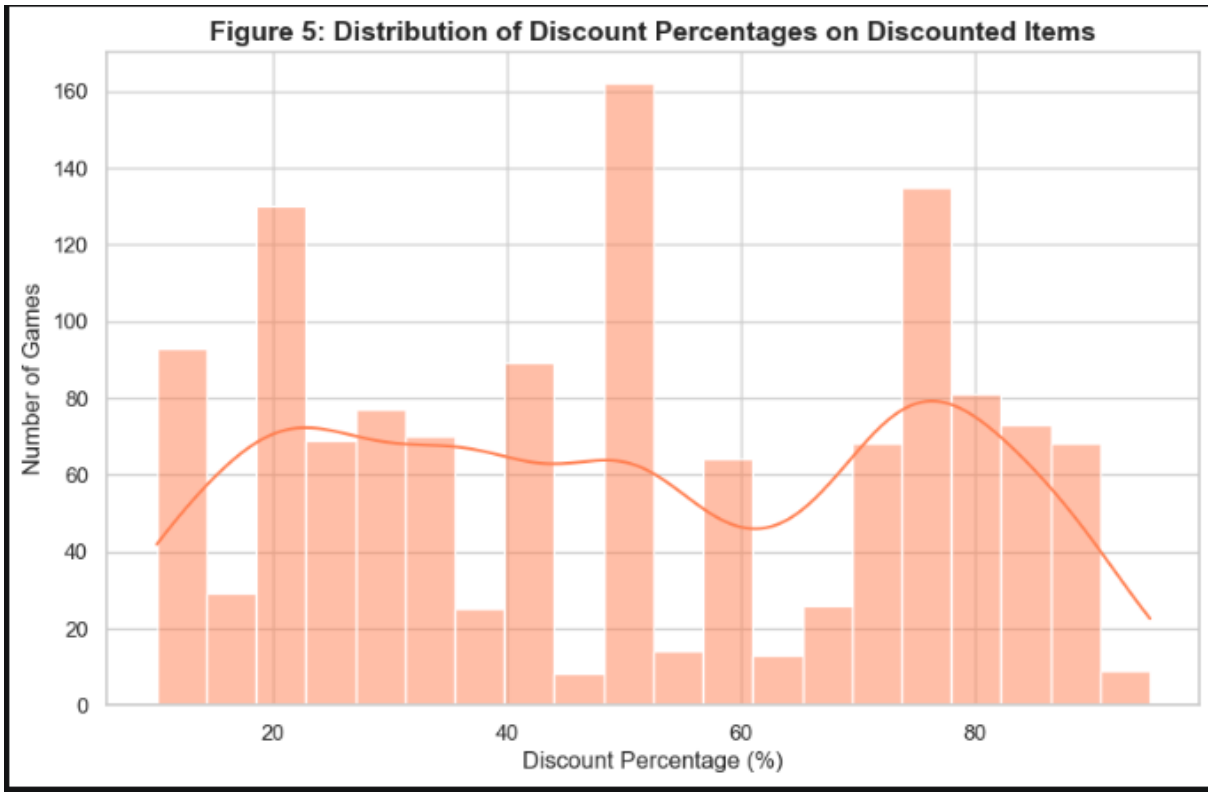
A bar chart or pie chart can compare records with a current price of zero against records with a positive current price.

This analysis should use the cleaned price field and exclude records with missing prices from the classification.

The initial project notes reported 451 records with a price of zero out of 3,377 raw records. This represents approximately 13.4% of the raw observations. However, this figure should be recalculated from the final cleaned dataset before it is presented as a final result.

It is also important to distinguish between the number of records and the number of unique games. The same game may appear in multiple daily observations, so the percentage of records representing free games may differ from the percentage of unique games that are free.

5.3 Discount Distribution



A histogram or boxplot can be used to examine the distribution of calculated discount percentages.

This analysis should include only records with valid calculated discounts. Records with missing original prices or other insufficient information should not be assigned assumed discount values.

The mean, median, and range of valid discounts should be calculated from the actual cleaned dataset. The results can then be used to identify common discount levels and unusually large price reductions.

5.4 Review Summaries

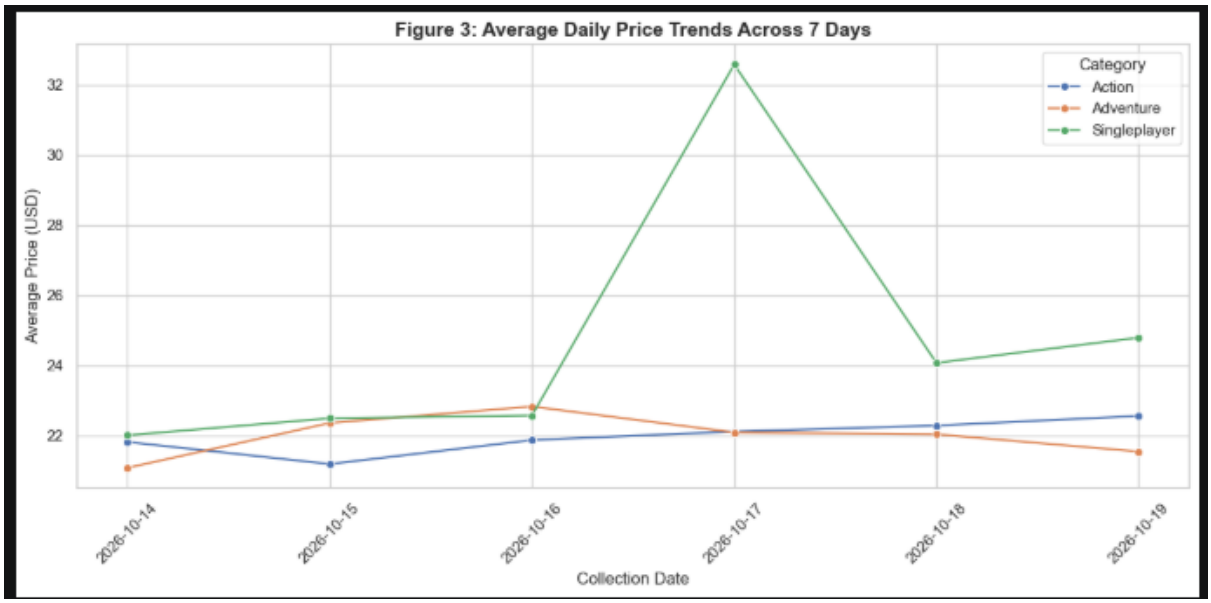
Suggested figure: 04_review_summaries.png

A bar chart can show the number of records in each review-summary category, such as "Very Positive", "Mostly Positive", and "Mixed".

Missing review summaries should be excluded from the category counts or displayed as a separate missing-data category.

The chart can describe the review summaries associated with the collected games. However, it cannot establish that positive reviews cause higher rankings without additional analysis and evidence.

5.5 Average Prices by Collection Date



A line chart can show how average recorded prices changed across the seven collection dates.

To produce a fair comparison, the analysis should use consistent price-cleaning rules and report the number of valid prices available for each date.

Average prices may change because the games collected on each date differ, because some prices change, or because promotional discounts affect the observations. Therefore, changes in the daily average should not automatically be interpreted as changes in the prices of the same games.

A stronger time-based analysis would track individual games across multiple collection dates.

5.6 Ranking Patterns

Suggested figure: 06_average_rank.png

A chart can compare the recorded ranks of games across collection dates or categories.

Lower numerical ranks represent higher positions in the ranked results. Comparing the same game's recorded rank across dates can help identify games whose positions remain relatively stable and those whose positions change.

The analysis should account for missing observations and differences between categories. The same game may not appear in every collection run.

The chart can describe observed ranking patterns, but it cannot establish the causes of those patterns from ranking data alone.

5.7 Research Question: Free and Paid Games

The project's research question is:

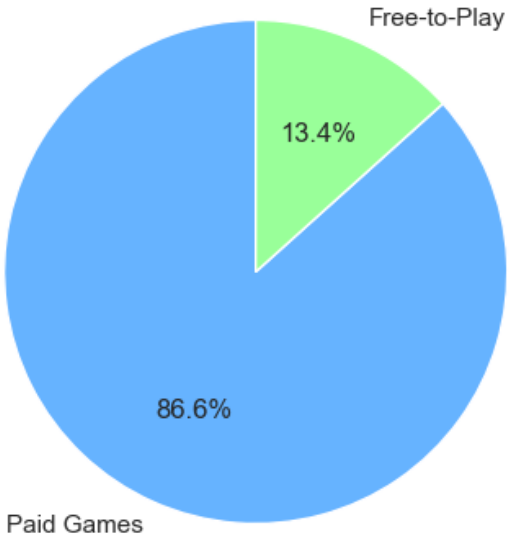
How do free-to-play games compare with paid games in representation and review sentiment within the collected Steam Top Sellers data?

To investigate this question, records can be classified as free or paid based on their cleaned current prices. Review summaries can then be grouped by price status and compared using counts or percentages.

The analysis should distinguish between observations and unique games because repeated daily records may influence the results. It should also account for missing prices and review summaries.

The findings can show whether free and paid games differ in their representation and recorded review sentiment within the collected sample. However, claims about review volume, player acquisition, or the effect of microtransactions would require additional variables and evidence.

Figure 6: Proportion of Free-to-Play vs. Paid Games in Top Sellers



6. RESPONSIBLE WEB SCRAPING

Responsible web scraping is important because automated requests can affect website services and must comply with applicable rules. The project identified the following practices as relevant to responsible data collection:

1. **Checking website rules:** The scraper was intended to inspect Steam's robots.txt file before collection. A DNS resolution error occurred during development, so the successful retrieval and inspection of the file must be confirmed before claiming that permissions were verified.
2. **Using an identifying User-Agent:** The scraper was configured to use a User-Agent identifying the project and course. A suitable contact detail should be included where required by the project's rules.
3. **Limiting request frequency:** Requests should be spaced appropriately, with delays and retry procedures used to reduce unnecessary repeated requests.
4. **Handling network failures:** Temporary failures should be handled through exception handling and controlled retries rather than causing the program to terminate unexpectedly.
5. **Collecting public product information:** The project focused on public game metadata rather than personal user information or account credentials.
6. **Preserving collection records:** The original CSV files and timestamps should be retained to support verification and reproducibility.

These practices help make the collection process more transparent and reliable. However, the use of a User-Agent and request delays alone does not establish that all website rules have been satisfied.

7. CHALLENGES AND LIMITATIONS

7.1 Internet Connectivity and DNS Resolution

During development, the scraper encountered a DNS resolution error when attempting to access store.steampowered.com. The error indicated that the computer could not resolve the website's domain name.

This prevented the relevant request from completing. The appropriate response was to check the internet connection, test domain resolution, and retry the request after connectivity was restored.

The incident demonstrated the importance of network error handling in web scraping. It also means that successful inspection of robots.txt must be confirmed rather than assumed.

7.2 Incomplete Collection on 17 October

The 17 October collection produced 377 records instead of the intended 500.

The difference may have resulted from a network interruption, pagination problem, unavailable results, or another limitation. The exact cause cannot be established from the record count alone.

The collected records were retained rather than artificially adding 123 observations. Future versions of the scraper should check the number of records returned, report incomplete runs, and retry failed requests where appropriate.

7.3 Errors When Saving CSV Files

An OSError occurred during development when the program attempted to save scraped data.

Possible causes include an incorrect output path, a missing folder, or a file-access problem. The precise cause should be determined from the complete error message.

7.4 Missing and Inconsistent Price Information

Missing current and original prices created difficulties when calculating discount percentages. The original discount column was also entirely empty.

The cleaning approach was therefore revised to retain the discount field and calculate values only when the current and original prices supported a valid calculation.

This approach avoids replacing unknown values with assumptions and improves the reliability of subsequent analysis.

7.5 Repeated Game Observations

Games may appear on several collection dates or in different categories. Removing every repeated game identifier would therefore remove legitimate observations.

The project needed to distinguish exact duplicates from valid repeated observations. This is particularly important for analysing changes in prices and rankings over time.

7.6 Database Connection and Import

Preparing a dataset for MySQL requires a working local MySQL server, an appropriate Python database driver, correct connection settings, and a successful import procedure.

The database and table must be verified after the import. If the table does not exist or contains fewer records than expected, the import process must be investigated before database storage can be reported as complete.

7.7 Limitations of the Analysis

The project has several limitations.

First, the data represent selected collection times rather than continuous observations of Steam. Prices and rankings may change at other times.

Second, prices can vary by region and currency. The results therefore reflect the storefront information captured during collection.

Third, the incomplete collection on 17 October means that the number of observations differs across dates.

Fourth, missing original prices limit the number of records for which discounts can be calculated.

Finally, repeated observations of the same game can influence summary statistics if they are treated as independent games. Future analysis should distinguish between unique games and daily observations.

8. RECOMMENDATIONS

Based on the work undertaken, the following recommendations are made.

First, improve the scraper's reliability. Network errors, failed requests, and incomplete collection runs should be handled using controlled retries, suitable delays, and clear error messages.

Second, validate the number of records collected. Each run should report the target number of records and the actual number collected. Incomplete runs should be documented and investigated.

Third, preserve raw data. The original CSV files should remain unchanged so that cleaning decisions can be reviewed and the final dataset can be recreated when necessary.

Fourth, improve missing-value handling. Missing prices, original prices, and review summaries should be handled according to their meanings rather than replaced with unsupported estimates.

Fifth, validate discount calculations. Only records with suitable current and original prices should receive calculated discounts. Invalid or inconsistent prices should be investigated.

Sixth, verify database storage. After importing the cleaned dataset, compare the record count in MySQL with the final cleaned dataset and inspect sample records to confirm that the import succeeded.

Seventh, strengthen the analysis. Future work should distinguish unique games from repeated daily observations and use consistent definitions when comparing categories, prices, discounts, and rankings.

Finally, document all findings accurately. Statistical claims and charts should be generated from the actual cleaned dataset. This will make the report more reliable and easier to reproduce.

9. CONCLUSION

This project applied data mining and data wrangling techniques to Steam Top Sellers information collected across seven dates, from 13 October to 19 October 2026. The seven raw CSV datasets contained 3,377 records, including 500 records from six collection runs and 377 records from the 17 October run.

The project involved merging the datasets using Python, inspecting data quality, identifying missing values, cleaning price fields, and preparing discount calculations. The initial inspection identified nine missing current prices, 2,074 missing original prices, 103 missing review summaries, and an entirely empty discount percentage column.

A key improvement was the decision to retain the discount percentage field and calculate values only when the available price information supported the calculation. The project also recognised the importance of preserving valid repeated observations from different dates while removing exact duplicates where appropriate.

The cleaned dataset provides a foundation for analysing price distributions, discounts, review summaries, free and paid games, and ranking patterns. MySQL was selected as the relational database for storing the prepared data, but the final import must be verified in the local environment before successful storage can be confirmed.

Overall, the project provided practical experience in web scraping, data cleaning, Python programming, relational database management, and exploratory data analysis.

REFERENCES

1. Beautiful Soup. *Beautiful Soup Documentation*. Available at: https://www.crummy.com/software/BeautifulSoup/bs4/doc/
2. Matplotlib Development Team. *Matplotlib Documentation*. Available at: https://matplotlib.org/stable/
3. MySQL. *MySQL Reference Manual*. Available at: https://dev.mysql.com/doc/
4. pandas Development Team. *pandas Documentation*. Available at: https://pandas.pydata.org/docs/
5. Python Software Foundation. *Python Documentation*. Available at: https://docs.python.org/3/
6. Valve Corporation. *Steam Store*. Available at: https://store.steampowered.com/

APPENDIX A: BASIC DATA-MERGING CODE

The following example shows how seven CSV files can be combined. Update the folder path and filenames to match the files used in the project.

```
import pandas as pd
from pathlib import Path

data_folder = Path("data") # Folder containing the seven CSV files.

csv_files = sorted(data_folder.glob("*.csv"))   # Find CSV files in the folder.

dataframes = []  # Read each file and record its source.

for file in csv_files:
    data = pd.read_csv(file)
    data["source_file"] = file.name
    dataframes.append(data)

df = pd.concat(dataframes, ignore_index=True)  # Combine all files.

# Save the merged dataset.
df.to_csv(
    data_folder / "steam_merged.csv",
    index=False)

print("Files merged:", len(csv_files))  # Display basic information.
print("Total records:", len(df))
print(df.head())
```

APPENDIX B: MYSQL VERIFICATION

Run the following SQL statements in MySQL Workbench after importing the cleaned dataset:

```
USE steam_data;


-- Check the table structure.

DESCRIBE steam_top_sellers_cleaned;

-- Check the total number of records.

SELECT COUNT(*) AS total_records

FROM steam_top_sellers_cleaned;


-- View a sample of the data.

SELECT *

FROM steam_top_sellers_cleaned

LIMIT 10;
```